SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1220	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Industry Problem solving task
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	ASWIN M	**Reg. No.:**	19241437
Section / Batch:	B.E (ECE) 3rd Year	**Date:**	18/08/2026

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A high-frequency financial trading system must process large volumes of market data with extremely low response time. Analyze the roles of CPU caches, DRAM, and persistent storage in handling frequently accessed trading data. Considering latency, data locality, capacity, and bandwidth, propose a memory hierarchy that minimizes data-access delays and supports high transaction throughput.
2. A medical imaging workstation processes large CT and MRI datasets while simultaneously running image-enhancement and visualization applications. Evaluate the interaction between cache memory, DRAM, and high-capacity storage during image processing. Propose an optimized memory hierarchy that balances access latency, memory capacity, bandwidth, and cost to improve overall processing performance.
3. A smart surveillance system receives continuous video streams from multiple high-resolution cameras and must detect objects in real time. Analyze the memory requirements for storing video frames, intermediate processing data, and frequently accessed detection parameters. Compare cache, DRAM, and non-volatile memory technologies and design a hierarchy that minimizes memory bottlenecks while maintaining high data throughput.
4. A scientific computing system performs large-scale simulations involving matrices and continuously accessed numerical datasets. Analyze how cache capacity, data locality, cache misses, DRAM bandwidth, and memory latency affect the execution of computational kernels. Recommend memory-hierarchy improvements that can reduce cache misses, increase effective memory bandwidth, and improve overall simulation performance.
5. A wearable health-monitoring device continuously collects sensor readings while operating under strict battery constraints. Analyze the trade-offs among SRAM, DRAM, and non-volatile memory in terms of latency, capacity, energy consumption, and data retention. Design a memory hierarchy that provides rapid access to frequently processed sensor data while minimizing power consumption and extending battery life.

Code of Conduct

I ASWIN M - 192414237*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10		/ 100

Course Coordinator **Faculty Incharge**

Question 1

Name : Aswin M
Reg No: 192412437
Subject Code: CSA1220

Memory Hierarchy Optimization for High Frequency Trading Systems

Objective

To design an efficient memory hierarchy for a high frequency trading system that provides very low memory access latency, high bandwidth, and fast processing of continuously changing market data.

Proposed Memory Hierarchy

A high frequency trading system requires frequently accessed data to be placed as close to the processor as possible. The proposed hierarchy is:

1. CPU Registers
2. L1 Cache
3. L2 Cache
4. L3 Cache
5. DRAM
6. SSD

Importance of Data Locality

Data locality is important because HFT applications repeatedly access the same or related data.

Temporal locality: means recently accessed data is likely to be accessed again. For example, the current price of a frequently traded stock can remain in cache.

Spatial locality: means nearby memory locations are likely to be accessed together. An order book can therefore be organized so that related price and quantity information is stored close together.

Improving locality increases cache hit rates and reduces expensive accesses to DRAM.

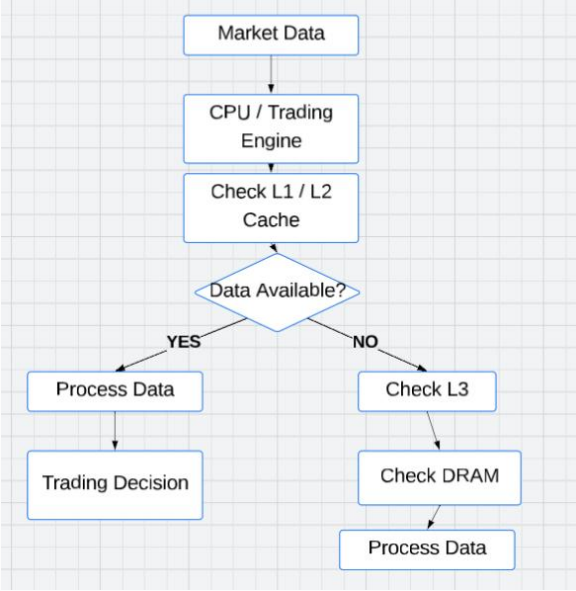
Performance Optimization Techniques

The following techniques can improve the memory performance of the HFT system:

- Increase cache hit rate by keeping frequently used market data in cache.
- Improve data locality by arranging related data close together in memory.
- Reduce unnecessary memory transfers between cache and DRAM.
- Keep critical trading variables in registers or L1 cache whenever possible.
- Use efficient data structures to reduce memory footprint and cache misses.
- Use high-bandwidth memory to support continuous market-data processing.

Latency and Bandwidth Optimization

The system should prioritize latency-sensitive information in the upper levels of the hierarchy.



Performance Analysis

Memory Level	Access Speed	Capacity	Main Use
Registers	Very High	Very Low	Active calculations
L1 Cache	Very High	Low	Critical data
L2 Cache	High	Medium	Frequently used data
L3 Cache	Moderate	Higher	Shared data
DRAM	Lower	High	Working dataset
SSD	Low	Very High	Historical data

The hierarchy provides a balance between speed and capacity. Frequently accessed information is placed in faster memory, while large and less frequently accessed datasets are moved to lower levels.

Result

The proposed memory hierarchy significantly improves the suitability of the system for high frequency trading. By keeping critical market data close to the processor, exploiting temporal and spatial locality, and reducing unnecessary DRAM and storage accesses, the system can achieve lower memory latency, higher bandwidth utilization, and faster trading decisions.

Question 2

Optimized Memory Hierarchy for Medical Image Processing

Objective

To design an optimized memory hierarchy for a medical image processing system that can efficiently handle large image datasets while providing high memory bandwidth, low access latency, and fast processing of medical images.

Role of Each Memory Level

CPU Registers:
Registers store intermediate values generated during image-processing calculations. Pixel values, filter coefficients, and temporary arithmetic results can be held in registers during processing.

L1 Cache:
L1 cache stores the most frequently accessed image blocks and processing instructions. Since image-processing algorithms repeatedly operate on nearby pixels, L1 cache can significantly reduce memory access time.

L2 Cache:
L2 cache provides additional space for recently accessed image blocks. It is useful when the working data is larger than the L1 cache.

L3 Cache:
L3 cache provides a larger shared cache for multiple processing cores. It can store frequently accessed portions of large medical images and support parallel image-processing tasks.

DRAM:

DRAM stores the active medical images being processed. MRI and CT images can contain large numbers of pixels or three-dimensional voxels, making high-capacity and high-bandwidth main memory important.

SSD:

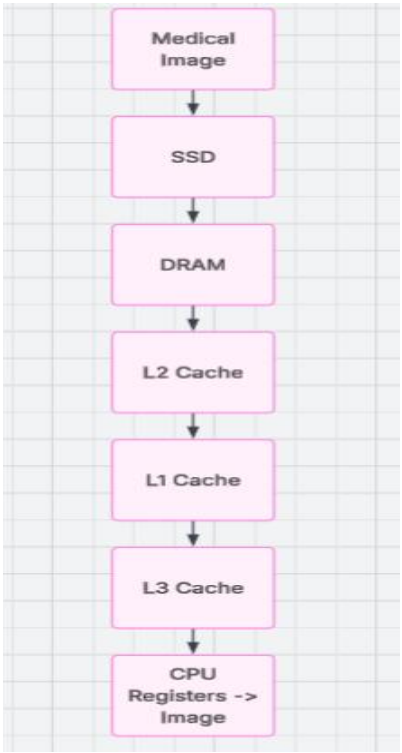
SSD storage is used for the medical image database and long-term storage of patient scans. Since SSD access is slower than cache and DRAM, images should preferably be loaded into DRAM before intensive processing.

Importance of Spatial Locality

Medical image processing makes extensive use of **spatial locality**. When one pixel is accessed, nearby pixels are usually required for operations such as filtering, edge detection, segmentation, and convolution.

For example, a convolution operation may process a small region around each pixel:

Memory Access Flow



The image is first loaded from persistent storage into DRAM. Frequently accessed portions are then brought into the cache hierarchy for faster processing.

Performance Optimization

The memory hierarchy can be optimized using the following techniques:

1. **Cache blocking:** Divide large images into smaller blocks that fit efficiently into cache.
2. **Sequential memory access:** Process pixels in an organized order to improve cache utilization.
3. **Prefetching:** Load image data into cache before it is required.
4. **High-bandwidth DRAM:** Provide sufficient bandwidth for large image transfers.
5. **Parallel processing:** Allow multiple CPU cores to process different image regions simultaneously.
6. **Efficient data representation:** Use appropriate pixel formats to reduce unnecessary memory usage.

Memory Level	Primary Purpose
Registers	Intermediate calculations
L1 Cache	Frequently processed pixels
L2 Cache	Recently used image blocks
L3 Cache	Shared processing data
DRAM	Active medical images
SSD	Medical image database

The main performance challenge is transferring large medical images between DRAM and the processor. Efficient caching and blocking techniques reduce this overhead and allow the processor to spend more time performing actual image-processing operations.

Result

The proposed memory hierarchy improves medical image processing by keeping frequently accessed image regions close to the CPU and storing larger datasets in DRAM and SSD storage. Cache blocking, spatial locality, prefetching, high memory bandwidth, and parallel processing help reduce memory access delays and improve overall processing performance. This architecture is suitable for applications such as MRI reconstruction, CT image enhancement, image segmentation, and medical diagnosis.

Question 3

Memory Hierarchy Design for Real Time Smart Surveillance

Objective

A real time smart surveillance system continuously receives video from one or more cameras and processes the frames to identify events such as motion, people, vehicles, or unusual activity. Since video frames arrive continuously, the processor must access image data quickly enough to prevent frame loss and processing delays.

The memory hierarchy should therefore be designed around three major requirements:

Low latency for immediate video processing
High bandwidth for continuous frame transfers
Large capacity for storing multiple frames and recorded footage

Memory Organization

The memory system can be divided according to the importance and lifetime of the data.

Fastest memory – Registers and L1 Cache

The processor uses these memories for immediate calculations. Pixel values, coordinates, comparison results, and temporary variables can be accessed with very low latency.

Intermediate memory – L2 and L3 Cache

These levels hold frequently reused portions of video frames and data required by object-detection algorithms. Shared L3 cache can also allow multiple processor cores to access common surveillance information.

Main memory – DRAM

DRAM stores complete frames and multiple frame buffers. It acts as the main workspace between the camera input and processing unit.

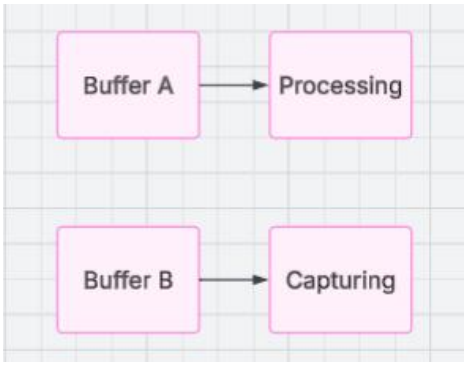
Large storage – SSD

The SSD stores recorded video, event information, detected-object data, and footage required for later investigation.

Double Buffering Technique

A useful technique for real-time surveillance is double buffering.

Two buffers are maintained in DRAM:



While the processor analyzes Buffer A, the camera stores the next frame in Buffer B. After processing is completed, their roles are exchanged.

Frame 1 → Buffer A → Processing

Frame 2 → Buffer B → Capturing

Frame 3 → Buffer A → Capturing

Frame 2 → Buffer B → Processing

Cache Utilization

Smart surveillance algorithms repeatedly access nearby image data. For example, an object detection algorithm may examine a region of several neighboring pixels.

Performance Considerations

The major performance factors are:

Memory bandwidth:
A high bandwidth memory system is required because every second of video can contain many frames and a large number of pixels.

Latency:
Critical processing data should remain in cache so that the processor does not repeatedly access slower DRAM.

Buffer capacity:
Sufficient buffer space prevents incoming frames from being lost when processing temporarily takes longer.

Cache efficiency:
Processing nearby pixels together improves spatial locality and reduces cache misses.

Parallelism:
Multiple CPU or GPU processing units can analyze different frames or image regions simultaneously.

Assume a camera produces:

- 30 frames per second
- Each frame requires 8 MB of memory
- The raw incoming data rate is:

$30 \times 8 = 240$ MB/s

Therefore, the memory subsystem must be capable of continuously handling at least this incoming data rate, in addition to the extra memory traffic generated during image processing.

If several cameras are connected, the required bandwidth increases accordingly.

Question 4

Memory Hierarchy Optimization for Scientific Computing

Introduction

Scientific computing applications such as weather prediction, climate modeling, physics simulations, and numerical analysis perform a large number of calculations on huge datasets. The processor often needs to repeatedly access matrices, vectors, simulation values, and intermediate results. Therefore, an efficient memory hierarchy is required to reduce processor waiting time and improve computational throughput.

Proposed Memory Organization

The memory system can be organized according to the size and frequency of data access:

Working Set of Scientific Applications

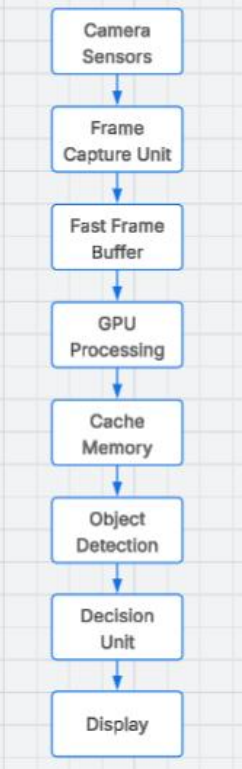
Scientific programs commonly operate on arrays and matrices. Consider a matrix multiplication:

C = A X B

The processor repeatedly accesses elements of matrices A, B, and C. If these elements are repeatedly fetched from DRAM, memory latency can become a major performance bottleneck.

Instead, the matrices can be divided into smaller blocks.

Proposed Architecture



Importance of Cache Blocking

Cache blocking improves performance by keeping a small working set of matrix elements inside the cache.

Once the computation for a block is completed, the processor moves to the next block.

This reduces the number of slow DRAM accesses and improves cache reuse.

Memory Bandwidth

Scientific applications can require very high memory bandwidth because large amounts of numerical data are continuously transferred between memory and processing units.

The performance can be limited by either:

Computation speed, when the processor is busy performing calculations.
Memory bandwidth, when the processor waits for data.

A well-designed hierarchy attempts to keep the processor supplied with data so that computational resources remain active.

Optimization Techniques

1. Cache Blocking:
Large matrices and arrays are divided into smaller blocks that fit into cache.

2. Data Prefetching:
Data likely to be required soon can be loaded into cache before it is requested.

3. Sequential Access:
Arrays should preferably be accessed in an order that makes efficient use of cache lines.

4. Parallel Processing:
Large datasets can be divided among multiple processing cores.

5. Efficient Data Structures:
Reducing unnecessary data movement decreases memory traffic and improves performance.

Result

An optimized memory hierarchy for scientific computing should combine fast registers and caches with high-capacity DRAM and persistent storage. Cache blocking, data locality, prefetching, and parallel processing reduce memory access delays and improve computational efficiency.

The proposed design is particularly suitable for matrix calculations, simulations, numerical modeling, climate analysis, and large-scale scientific workloads, where efficient movement of data is as important as the computational power of the processor.

Question 5

Energy Efficient Memory Hierarchy for Wearable Health Monitoring

Introduction

Wearable health monitoring devices continuously collect information from sensors such as heart rate, body temperature, blood oxygen level, and physical activity sensors. Since these devices are battery powered and have limited processing and memory resources, the memory hierarchy must provide low power consumption, low latency, and sufficient storage capacity.

The main objective is to reduce unnecessary memory accesses because frequent access to slower memory consumes more energy.

Proposed Memory Organization

Sensors

↓

Microcontroller

↓

Registers

↓

L1 Cache

↓

Low Power RAM

↓

Flash / EEPROM

↓

Mobile / Cloud

The hierarchy places frequently processed sensor data in fast memory while storing less frequently accessed information in larger non-volatile memory.

Memory Usage in the Wearable Device

Registers

Registers store temporary values during sensor-data processing. For example, the current heart-rate value or temperature measurement can be held in registers while calculations are performed.

L1 Cache

The cache stores frequently accessed instructions and sensor values. Keeping active data in cache reduces the need to access larger memories repeatedly.

Low Power RAM

RAM acts as the working memory for the wearable device. It can store recent sensor readings, filtering results, and temporary buffers.

Flash / EEPROM

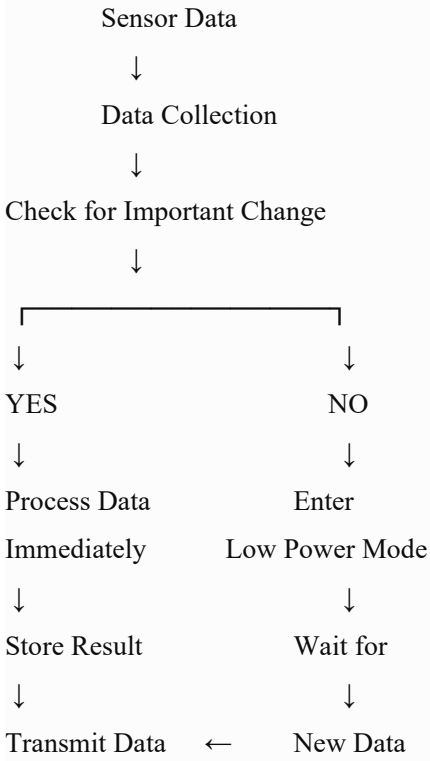
Non-volatile memory stores information that must remain available after the device is powered off. Examples include device settings, patient history, and stored measurements.

Mobile or Cloud Storage

Long-term health records can be transferred to a smartphone or cloud platform. This allows the wearable device to avoid maintaining a very large local dataset.

Energy Saving Strategy

A major design principle is to keep the processor and memory active only when necessary.



For example, if the measured heart rate remains within a normal range, the device does not need to perform intensive processing continuously. The processor can operate at a lower power state and wake up when new data arrives or when an abnormal value is detected.

Data Reduction

Instead of continuously storing every raw sensor reading, the device can perform basic processing locally.

Memory Access Optimization

The following techniques can improve energy efficiency:

1. Local Processing:
Basic filtering and analysis should be performed locally to reduce unnecessary data transfers.

2. Low Power RAM:
Low-power memory should be used for frequently accessed sensor data.

3. Sleep Modes:

Unused memory and processor components should enter low-power states.

4. Data Compression:

Sensor information can be compressed before storage or transmission.

5. Selective Data Storage:

Only important measurements and detected events need to be stored for long periods.

6. Efficient Buffering:

Sensor readings can be collected in small buffers and transferred together instead of performing frequent memory operations.

Performance and Energy Trade-Off

There is a trade-off between memory speed, capacity, and energy consumption.

Memory	Speed	Capacity	Energy Requirement
Registers	Very High	Very Low	Very Low
L1 Cache	High	Low	Low
Low Power RAM	Moderate	Medium	Low
Flash / EEPROM	Lower	High	Low for retention
Cloud Storage	Dependent on network	Very High	Higher during transmission

The wearable should use the fastest memory only when required and keep larger or less frequently accessed data in lower-power storage.

Result

An energy-efficient memory hierarchy for wearable health monitoring should combine registers, cache, low-power RAM, non-volatile memory, and external storage. Local processing, selective data storage, buffering, compression, and sleep modes can significantly reduce unnecessary memory accesses and energy consumption.

The proposed architecture provides a balance between processing speed, memory capacity, and battery life, making it suitable for wearable devices that continuously monitor health parameters while operating for long periods on limited battery power.